

COMP6237 Data Mining

Lecture 1: Recommendation Systems

Zhiwu Huang

Zhiwu.Huang@soton.ac.uk

Lecturer (Assistant Professor) @ VLC of ECS
University of Southampton

Lecture slides available here:

<http://comp6237.ecs.soton.ac.uk/zh.html>

[Book time with Zhiwu Huang: Office Hour](#)

(Thanks to Prof. Jonathon Hare and Dr. Jo Grundy for providing the lecture materials used to develop the slides.)

My Lectures

17-Feb	4	Zhiwu	Making Recommendations
18-Feb		Zhiwu	Finding Groups
20-Feb		Zhiwu	Covariance
24-Feb	5	Zhiwu	Embedding Data
25-Feb		Zhiwu	Search
27-Feb		Zhiwu	Document filtering
03-Mar	6	Zhiwu	Modelling with decision trees
04-Mar		Zhiwu	Modelling Prices & Nearest Neighbours
06-Mar		Zhiwu	Market Basket Analysis
10-Mar	7	Zhiwu	Semantic Spaces & Latent Semantics
11-Mar		Zhiwu	Topic Modelling
13-Mar		Zhiwu	Outlier Detection

Four Sets of Key Slides for Each Lecture:

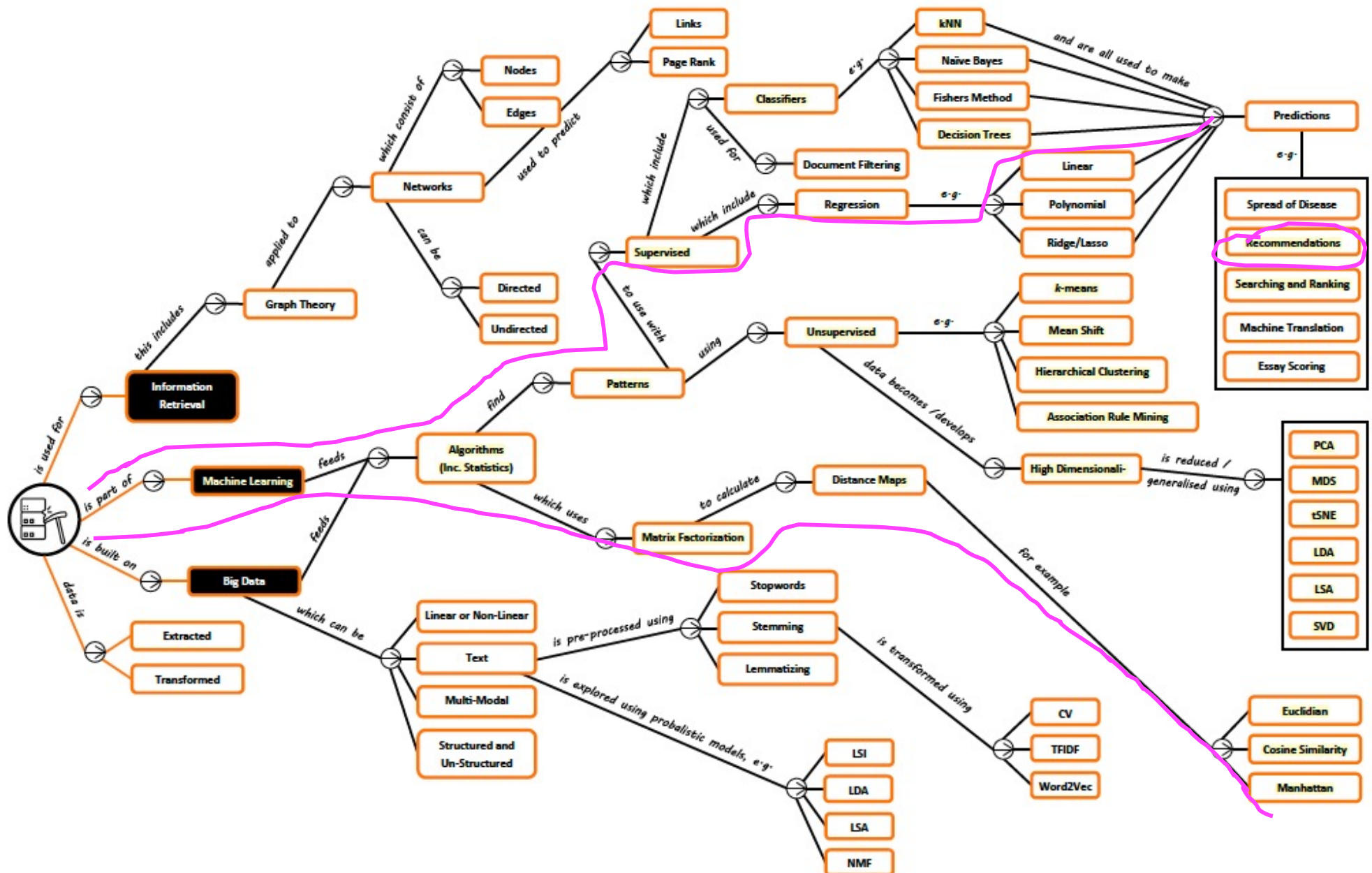
Roadmap + Textbook + Overview + Learning Outcomes (Exercise/Exam)

<http://comp6237.ecs.soton.ac.uk/zh.html>

AICE1006-EDA for Data Processing: <https://github.com/zhiwu-huang/AICE1006-Data-Analytics-EDA>

COMP6237 for Deep Learning: <https://secure.ecs.soton.ac.uk/module/2425/COMP6252/43095/>

Recommendation Systems – Roadmap



Recommendation Systems – Textbook

Chapter 9

Recommendation Systems

There is an extensive class of Web applications that involve predicting user responses to options. Such a facility is called a *recommendation system*. We shall begin this chapter with a survey of the most important examples of these systems. However, to bring the problem into focus, two good examples of recommendation systems are:

1. Offering news articles to on-line newspaper readers, based on a prediction of reader interests.
2. Offering customers of an on-line retailer suggestions about what they might like to buy, based on their past history of purchases and/or product searches.

► Mining of Massive Datasets J. Leskovec *et al*

Recommendation Systems – Overview (1/4)

Cast & crew · User reviews · Trivia · FAQ · IMDbPro · All topics

Migration

2023 · PG · 1h 23m

IMDb RATING 6.7/10 16K

YOUR RATING 9/10

POPULARITY 49 11



Odd ducks welcome

ILLUMINATION PRESENTS
MIGRATION
Only In Theaters This Holiday



Play trailer 2:25

16 VIDEOS

99+ PHOTOS

Animation Adventure Comedy

A family of ducks try to convince their overprotective father to go on the vacation of a lifetime.

Directors Benjamin Renner · Guylo Homsy

Writers Mike White · Benjamin Renner

Stars Kumail Nanjiani · Tresi Gazal · Elizabeth Banks

IN THEATERS

see showtimes

Add to Watchlist
Added by 34.6K users

<https://www.imdb.com/title/tt6495056/>, Release date: 2 February 2024 ([UK](#))

Recommendation Systems – Overview (2/4)

Top picks >

TV shows and movies just for you



★ 7.1 ☆

Wonka

Watch options

▶ Trailer



★ 6.0 ☆

Trolls Band Together

Watch options

▶ Trailer



★ 7.0 ☆

Elemental

Watch options

▶ Trailer



★ 7.2 ☆

Percy Jackson and the Olympians

+ Watchlist

▶ Trailer



Recommendation Systems – Overview (3/4)

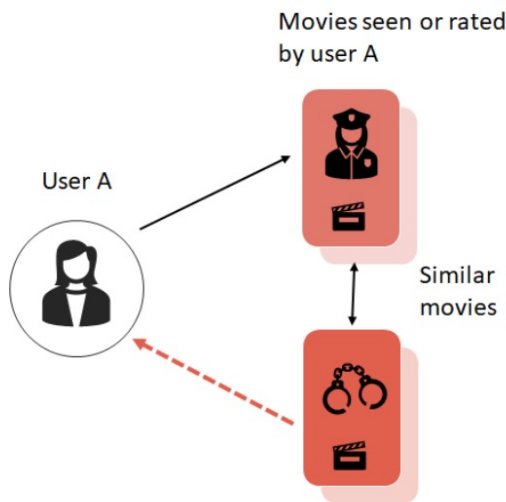
Film	Alice	Bob	Carol	Dave	x_1 romance	x_2 action
Love Really	4	1		4	1	0.1
Deadly Weapon		1	4	5	0.1	1
Fast and Cross	5		5	4	0.2	0.9
Star Battles	1	5			0.1	1
Dark Knight	5			?	0.1	1



https://en.wikipedia.org/wiki/The_Dark_Knight

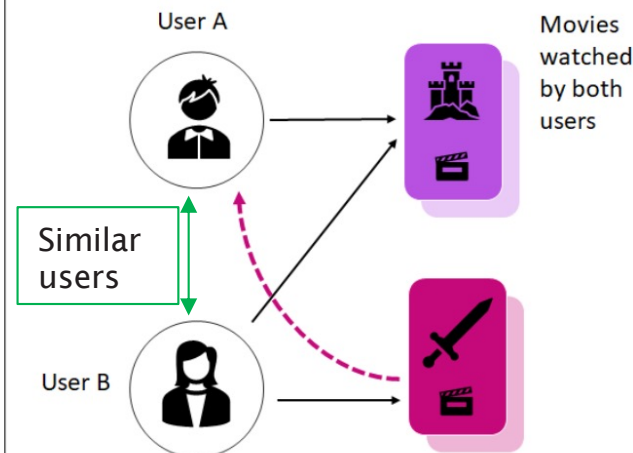
Content-based Filtering

Movie recommendations to user A



Collaborative Filtering

Movies watched by user A are recommended to user B



Remaining problems:

- How to represent the item/user data?
- How to save the data?
- How to compare the data (user/item)?

Recommendation Systems – Learning Outcomes

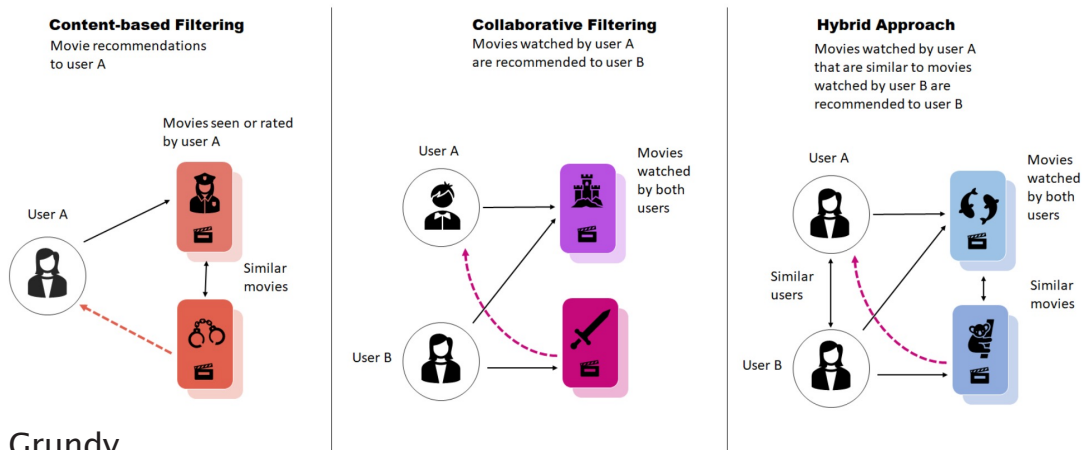
- **LO1:** Mastering fundamental concepts and mathematical calculations of content-based and user-based/item-based collaborative filtering approaches, such as (exam)
 - ❖ Measuring distances/similarities between users or items
 - ❖ Computing the resulting matrix using Compressed Row/Column Storage
 - ❖ Predicting the missing rating using content-based approach
 - ❖ Calculating the predicted rating with user-based collaborative filtering approach
- **LO2:** Implement basic algorithms using Python (coursework)

Assessment hints: Multi-choice Questions (single answer: concepts, calculation etc)

- *Textbook Exercises: textbooks (Programming + Mining)*
- *Other Exercises: <https://www-users.cse.umn.edu/~kumar001/dmbook/sol.pdf>*
- *ChatGPT or other AI-based techs*

Recommendation Systems – Algorithms

- **Content-based systems** (e.g., Netflix) *recommend items based on content features*. For instance, if a Netflix user has watched many cowboy movies, then recommend a movie classified in the database as having the “cowboy” genre.
- **Collaborative filtering systems** (e.g., Facebook Amazon) *recommend items based on similarity measures between users and/or items*. The items recommended to a user are those preferred by similar users.
- **Hybrid recommender systems** (e.g., Netflix) Uses combinations of different approaches



We will learn Content-based and collaborative filtering systems in this lecture

Recommendation Systems– Content based approach

Can use a vector of features for each film, eg romance, action *Content-based Features/data*

Film	Alice	Bob	Carol	Dave	x_1 romance	x_2 action
Love Really	4	1		4	1	0.1
Deadly Weapon		1	4	5	0.1	1
Fast and Cross	5		5	4	0.2	0.9
Star Fight	1	5			0.1	1

Each film can be described by the vector $X = [1 \quad x_1 \quad x_2]$ (1 is for the bias term)

Learn 2D parameter vector θ , where $\theta^T X$ gives the number of stars for each user.

$\theta = [0 \quad 5 \quad 0]$ for someone who really likes romance films

Recommendation Systems– Content based approach

Use **Linear Regression** to find user parameter vector θ where m is the number of films rated by that user

$$\min_{\theta} \frac{1}{m} \sum_{i=1}^m (\theta^T X_i - y)^2 \quad (1)$$

Can also use Bayesian classifiers, MLPs, etc.

Film	Alice	Bob	Carol	Dave	x_1 romance	x_2 action
Love Really	4	1		4	1	0.1
Deadly Weapon		1	4	5	0.1	1
Fast and Cross	5		5	4	0.2	0.9
Star Fight	1	5			0.1	1

Recommendation Systems– Content based approach

Linear Regression $\min_{\theta} \frac{1}{m} \sum_{i=1}^m (\theta^T X_i - y)^2$ (1)

Can use a vector of features for each film, eg romance, action

Film	Alice	Bob	Carol	Dave	x_1 romance	x_2 action
Love Really	4	1		4	1	0.1
Deadly Weapon		1	4	5	0.1	1
Fast and Cross	5		5	4	0.2	0.9
Star Fight	1	5			0.1	1

Take the user ‘Alice’ as an example

$$y_1 = 4, y_3 = 5, y_4 = 1$$

$$X_1 = [1, x_1, x_2] = [1, 1, 0.1], X_3 = [1, 0.2, 0.9], X_4 = [1, 0.1, 1],$$

$\theta = [b, w_1, w_2]$, what about y_2 for ‘Alice’?

Least Squares SSE $= [(\theta^T X_1 - y_1) + (\theta^T X_3 - y_3) + (\theta^T X_4 - y_4)] = \min_{\theta} \frac{1}{3} \sum_i (b + w^T X_i - y_i)$

Gradient Decent:

$$b = \mu_Y - w \cdot \mu_X$$

$$w = \frac{\sum_{i=1}^n (x_i - \mu_X)(y_i - \mu_Y)}{\sum_{i=1}^n (x_i - \mu_X)^2} = \frac{\sigma_{XY}}{\sigma_X^2} = \frac{\text{cov}(X, Y)}{\text{var}(X)}$$

Recommendation Systems– Content based approach

Use **Linear Regression** to find user parameter vector θ where m is the number of films rated by that user

$$\min_{\theta} \frac{1}{m} \sum_{i=1}^m (\theta^T X_i - y)^2 \quad (1)$$

Can also use Bayesian classifiers, MLPs, etc.

Problems?

requires hand coded knowledge of film

not easy to scale up

user may not have rated many films

Category films by genre

From sources across the web

Comedy	▼	Drama	▼	Horror	▼
Thriller	▼	Action	▼	Science fiction	▼
Fantasy	▼	Musicals	▼	Romance	▼
Western	▼	Crime	▼	Mystery	▼
Documentary	▼	Animation	▼	Basic movie genres	▼
Film noir	▼	Action comedy	▼	Animated documentary	▼
Biopic	▼	Buddy comedy	▼	Disaster	▼
Fantasy film	▼	Historical	▼	Historical films	▼

Can use a vector of features for each film, eg romance, action

Film	Alice	Bob	Carol	Dave	x_1 romance	x_2 action
Love Really	4	1		4	1	0.1
Deadly Weapon		1	4	5	0.1	1
Fast and Cross	5		5	4	0.2	0.9
Star Fight	1	5			0.1	1

Recommendation Systems – Collaborative Filtering

Collaborative Filtering example:

Alice likes Dr Who, Star Wars and Star Trek

Bob likes Dr Who and Star Trek

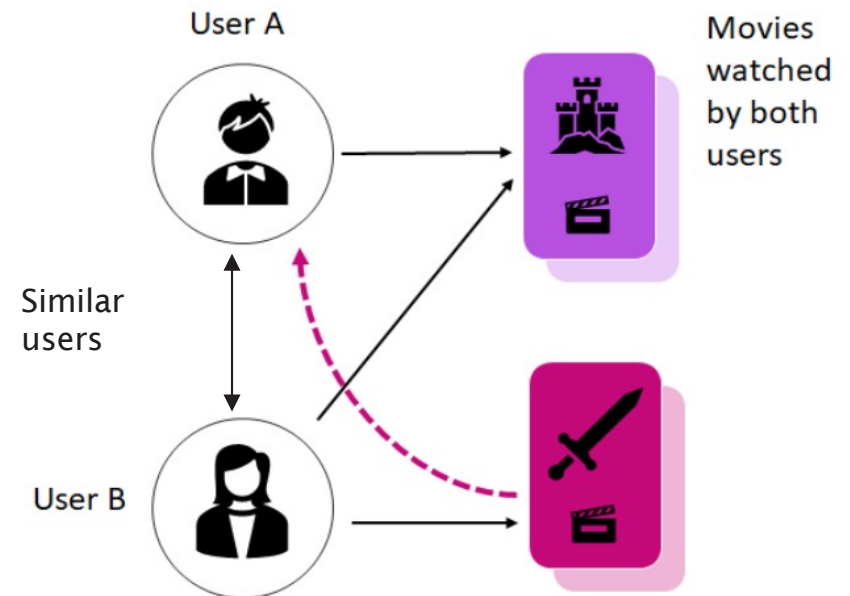
A recommender system would correlate the likes, and suggest that Bob might like Star Wars too.

Personal preferences can be correlated.

Task: Discover patterns in observed behaviour across a community of users

- ▶ Purchase history
- ▶ Item ratings
- ▶ Click counts

Predict new preferences based on those patterns



Recommendation Systems – Collaborative Filtering

Collaborative filtering uses a range of approaches to accomplish this task

- ▶ Neighbourhood based approach
- ▶ Model based approach
- ▶ Hybrid (Neighbourhood and model) based approach

This lecture will cover the Neighborhood based approach

Recommendation Systems – Collaborative Filtering

Measure user preferences. Eg. Film recommendation

Users rate films between 0 and 5 stars

	Life is Beautiful	Seven Samurai	Joker	Schindler's List	The Pianist	City of God
Lee	2.5	3.5	3.0	3.5	3.0	2.5
Sofia	3.0	3.5	1.5	5.0	3.0	3.5
Miley	2.5	3.0		3.5	4.0	
Justina		3.5	3.0	4.0	4.5	2.5
Donald	3.0	4.0	2.0	3.0	3.0	2.0
Mikey	3.0	4.0		5.0	3.0	3.5
Tristan		4.5		4.0		1.0

The data is **sparse**, there are missing values

Collaborative Filtering – Sparsity

Sparsity can be taken advantage of to speed up computations

Most libraries that do matrix algebra are based on LAPACK,
written in Fortran90

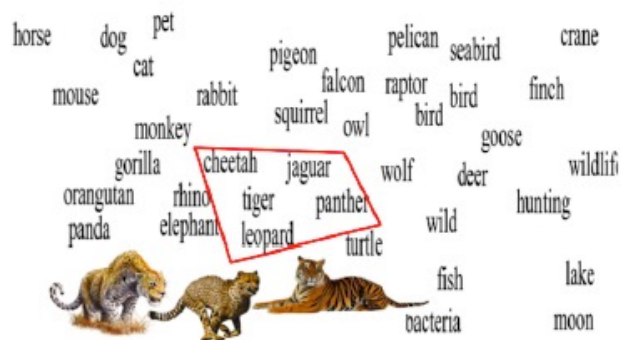
Computation is done by calls to the Basic Linear Algebra
Subprograms (BLAS).

This is how the Python `numpy` library does its linear algebra.

Collaborative Filtering – Feature Extraction

Features are stored in a 'feature vector', a fixed length list of numbers.

- ▶ The length of this vector is the number of dimensions
- ▶ Each vector represents a point and a direction in the featurespace.
- ▶ Each vector must have the same dimensionality



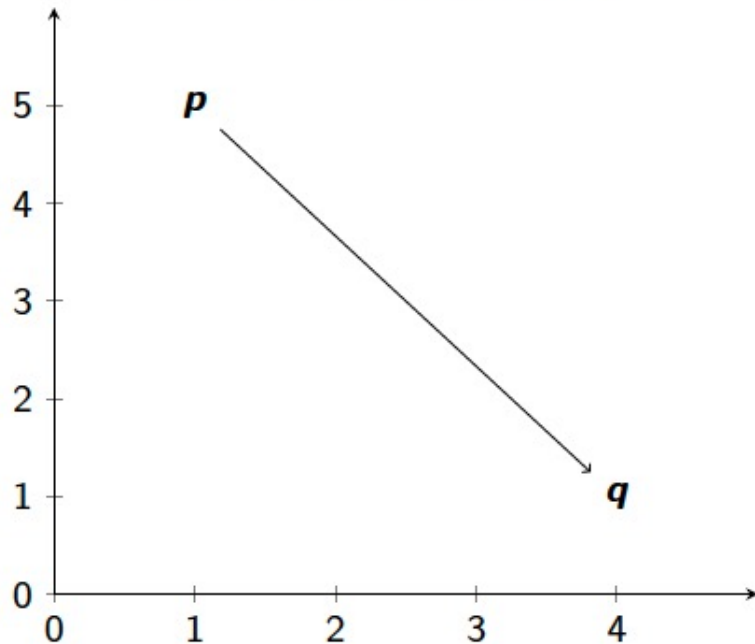
A projection of encoded word vectors shows that similar words are close to each other in feature space.

We say two things are *similar* if they have similar feature vectors, i.e. are close to each other in featurespace.

Collaborative Filtering – Distance

There are a number of ways to measure distance in feature space:

► Euclidean Distance:



$\mathbf{p}$ and $\mathbf{q}$ are N-dimensional feature vectors,

$$\mathbf{p} = [p_1, p_2, \dots, p_N],$$

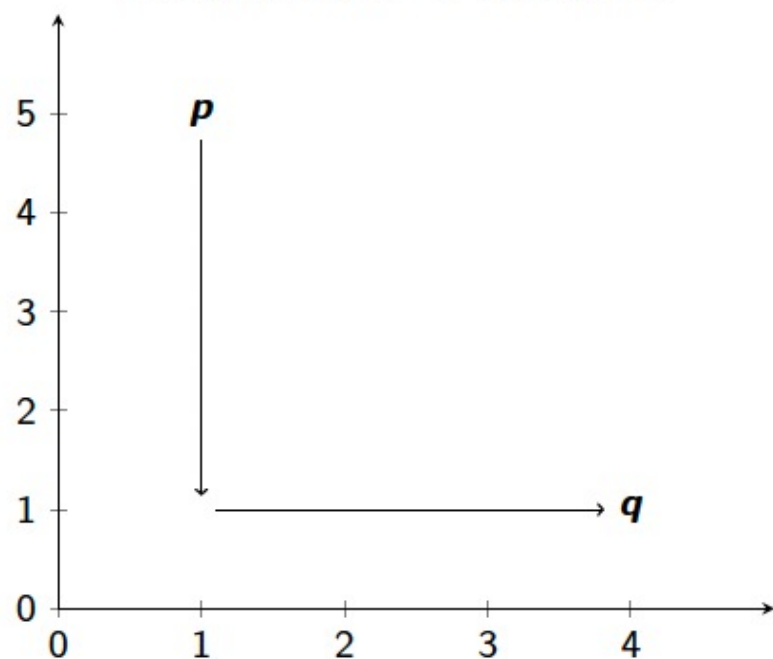
$$\mathbf{q} = [q_1, q_2, \dots, q_N]$$

Euclidean distance:

$$\|\mathbf{p} - \mathbf{q}\| = \sqrt{\sum_{i=1}^N (q_i - p_i)^2}$$

Collaborative Filtering – Distance

► Manhattan Distance:



$\mathbf{p}$ and $\mathbf{q}$ are N-dimensional feature vectors,

$$\mathbf{p} = [p_1, p_2, \dots, p_N],$$

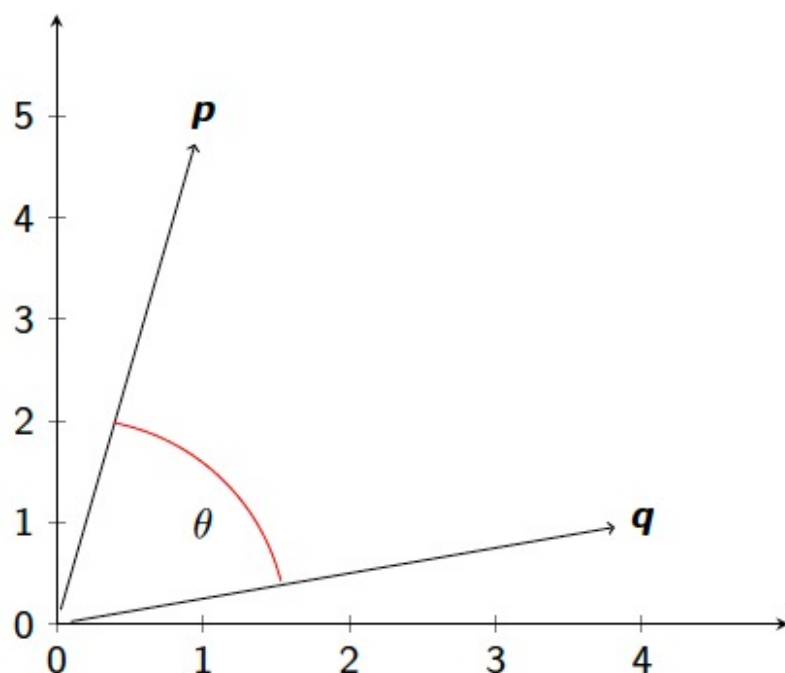
$$\mathbf{q} = [q_1, q_2, \dots, q_N]$$

Manhattan distance:

$$\|\mathbf{p} - \mathbf{q}\|_1 = \sum_{i=1}^N |q_i - p_i|$$

Collaborative Filtering – Distance

► Cosine Similarity



Only measures direction, not magnitude of vector.

p and ***q*** are N-dimensional feature vectors,

$$\mathbf{p} = [p_1, p_2, \dots, p_N],$$

$$\mathbf{q} = [q_1, q_2, \dots, q_N]$$

Cosine Similarity:

$$\cos(\theta) = \frac{\mathbf{p} \cdot \mathbf{q}}{|\mathbf{p}| |\mathbf{q}|}$$

$$= \frac{\sum_{i=1}^N p_i q_i}{\sqrt{\sum_{i=1}^N p_i^2} \sqrt{\sum_{i=1}^N q_i^2}}$$

Collaborative Filtering – User Similarity

Need to define a similarity score, based on the idea that similar users have similar tastes, i.e. like the same movies.)

Needs to take in to account sparsity, not all users have seen all movies.

Typically between 0 and 1, where 1 is the same, and 0 is totally different

Can visualise the users in feature space, using two dimensions at a time

Visualisation of users in film space ipynb

Film	Alice	Bob	Carol	Dave
Love Really	4	1		4
Deadly Weapon		1	4	5
Fast and Cross	5		5	4
Star Battles	1	5		

Collaborative Filtering – User Similarity

There are many ways to compute similarity based on Euclidean distance

We *could* chose:

$$sim_{L2}(x, y) = \frac{1}{1 + \sqrt{\sum_{i \in I_{xy}} (r_{x,i} - r_{y,i})^2}}$$

where $r_{x,i}$ is the rating from user x for item i

I_{xy} is set of items rated by both x and y

i.e. when the distance is 0, the similarity is 1, but when the distance is large, similarity $\rightarrow 0$

Collaborative Filtering – User Similarity

Can also use cosine similarity

$$sim_{cos}(x, y) = \frac{\sum_{i \in I_{xy}} r_{x,i} r_{y,i}}{\sqrt{\sum_{i \in I_{xy}} r_{xi}^2} \sqrt{\sum_{i \in I_{xy}} r_{yi}^2}}$$

Only performed over the items which are rated by both users

Collaborative Filtering – User Similarity

Alternatively, calculate correlation of users, based on ratings they share

Using Pearson's Correlation: standard measure of dependence between two related variables.

$$\text{sim}_{\text{Pearson}}(x, y) = \frac{\sum_{i \in I_{xy}} (r_{x,i} - \bar{r}_x)(r_{y,i} - \bar{r}_y)}{\sqrt{\sum_{i \in I_{xy}} (r_{x,i} - \bar{r}_x)^2 \sum_{i \in I_{xy}} (r_{y,i} - \bar{r}_y)^2}}$$

Where $\bar{r}_x$ is average rating user x gave for all items in I_{xy}

Correlation between users in ipynb

Collaborative Filtering – User Similarity

Users are inconsistent. Some users always give out 5s to films they like, whereas some are more picky.

For example, look at Lisa and Jill, both rank the films in the same order, but give different ratings.

Pearson correlation corrects for this, but Euclidean similarity doesn't.

Data normalisation and mean centering can overcome this.

Data standardisation

Film	Alice	Bob	Carol	Dave	Lisa	Jill
Love Really	4	1		4	1	2
Deadly Weapon		1	4	5	2	3
Fast and Cross	5		5	4	3	4
Star Battles	1	5			4	5

Collaborative Filtering – User Filtering

We now have a set of measures for computing the similarity between users

Produce a ranked list of best matches to a target user. Typically want the top- N users

May only want to consider a subset of users, i.e. those who rated a particular item.

Ranking users by similarity [ipynb](#)

Collaborative Filtering – Recommending

Now we have a list of similar users, how can we recommend items?
Predict rating $r_{u,i}$ of item i by user u as an aggregation of the ratings of item i by users similar to u

$$r_{u,i} = \text{aggr}_{\hat{u} \in U}(r_{\hat{u},i})$$

Where U is the set of *top* users most similar to u that rated item i
Multiply the score by the similarity of the user
Normalise by sum of similarities (otherwise items rated more often will dominate)

$$r_{u,i} = \frac{\sum_{\hat{u} \in U} \text{sim}(u, \hat{u}) r_{\hat{u},i}}{\sum_{\hat{u} \in U} |\text{sim}(u, \hat{u})|}$$

This is *User Based Filtering*

Demo: User based recommendation

Collaborative Filtering – User Based Filtering

Can also aggregate by computing average over similar users

$$r_{u,i} = \frac{1}{N} \sum_{\hat{u} \in U} r_{\hat{u},i}$$

Or by subtracting the average user rating score for all the items they scored, this is to compensate for people that judge generously or meanly.

$$r_{u,i} = \bar{r}_u + \frac{\sum_{\hat{u} \in U} \text{sim}(u, \hat{u})(r_{\hat{u},i} - \bar{r}_{\hat{u}})}{\sum_{\hat{u} \in U} |\text{sim}(u, \hat{u})|}$$

Recap – Recommendation Systems

Content-based

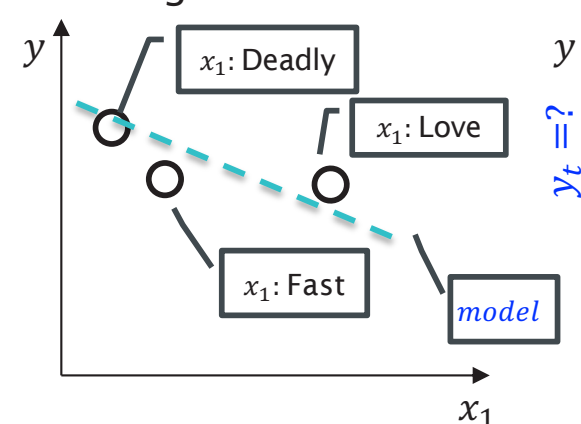
$$\min_{\theta} \frac{1}{m} \sum_{i=1}^m (\theta^T X_i - y)^2$$

$x(\text{Love}) = (x_1, x_2) = (1, 0.1)$

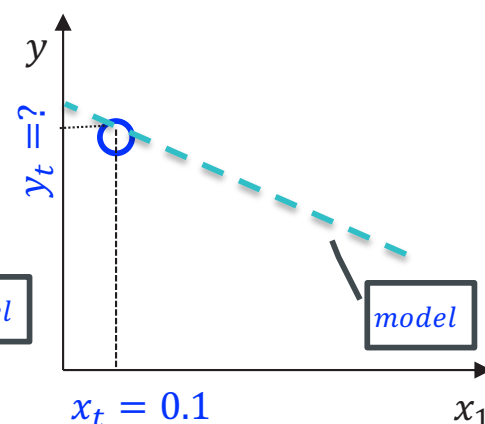
Film	Dave	x_1 romance	x_2 action
Love Really	4	1	0.1
Deadly Weapon	5	0.1	1
Fast and Cross	4	0.2	0.9
Star Battles	?	0.1	1

$y_t = ?$

1. Fitting User Behavior



2. Prediction



User-based Collaborative

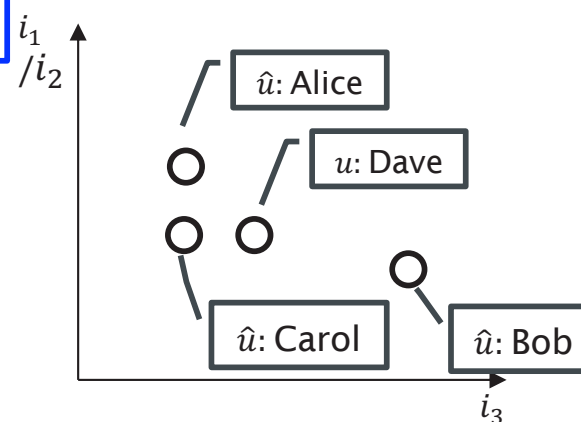
$$r_{u,i} = \frac{\sum_{\hat{u} \in U} \text{sim}(u, \hat{u}) r_{\hat{u},i}}{\sum_{\hat{u} \in U} |\text{sim}(u, \hat{u})|}$$

$\hat{u}(\text{Alice}) = (i_1, i_3) = (4, 5)$

	Film	Alice	Bob	Carol	Dave
i_1	Love Really	4	1		4
i_2	Deadly Weapon		1	4	5
i_3	Fast and Cross	5		5	4
	Star Battles	1	5	2	?

$r_{u,t} = ?$

1. Finding Similar Users



2. Prediction

Sim(Dave, Alice) = 0.8
Sim(Dave, Carol) = 0.6

$$r_{u,t} = \frac{0.8 \times 1 + 0.6 \times 2}{0.8 + 0.6}$$

e.g., Cosine Similarity:

$$\cos(\theta) = \frac{\mathbf{p} \cdot \mathbf{q}}{|\mathbf{p}| |\mathbf{q}|}$$

$$= \frac{\sum_{i=1}^N p_i q_i}{\sqrt{\sum_{i=1}^N p_i^2} \sqrt{\sum_{i=1}^N q_i^2}}$$

Note: The similarities (0.8, 0.6) above are merely used for demonstration. With Cosine similarity, they'd be 0.99 and 0.98 if we assume the missing ratings can be replaced with the user's average on other films excluding 'Star Battles' (e.g., "Deadly Weapon" with Alice is 4.5).